

MABATIROLLINGMILLSKE Catalogue Pagination Options

To scale your **MABATIROLLINGMILLSKE catalogue** for hundreds of mabati products, we can add **pagination** or **infinite scroll**. Here's how both options look in React + Tailwind:

Option 1: Pagination

Pagination.jsx

```
export default function Pagination({ currentPage, totalPages, onPageChange }) {
  return (
    <div className="flex justify-center space-x-2 mt-6">
      {Array.from({ length: totalPages }, (_, i) => (
        <button
          key={i}
          onClick={() => onPageChange(i + 1)}
          className={`px-3 py-1 rounded ${currentPage === i + 1 ? "bg-blue-600 text-white" : "bg-gray-200"}`}
        >
          {i + 1}
        </button>
      ))}
    </div>
  );
}
```

Update Catalogue.jsx

```

import { useState } from "react";
import ProductCard from "../ProductCard";
import ProductFilter from "../ProductFilter";
import PriceSort from "../PriceSort";
import Pagination from "../Pagination";

const products = [
  // ... full product list
];

export default function Catalogue() {
  const [filtered, setFiltered] = useState(products);
  const [currentPage, setCurrentPage] = useState(1);
  const itemsPerPage = 6;

  const handleFilter = ({ search, gauge, finish }) => {
    let result = products;
    if (search) result = result.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
    if (gauge) result = result.filter(p => p.gauge === gauge);
    if (finish) result = result.filter(p => p.finish === finish);
    setFiltered(result);
    setCurrentPage(1);
  };

  const handleSort = (order) => {
    let sorted = [...filtered];
    if (order === "asc") sorted.sort((a, b) => a.price - b.price);
    else if (order === "desc") sorted.sort((a, b) => b.price - a.price);
    setFiltered(sorted);
  };

  const totalPages = Math.ceil(filtered.length / itemsPerPage);
  const displayed = filtered.slice((currentPage - 1) * itemsPerPage, currentPage * itemsPerPage);

  return (
    <section className="p-6">
      <ProductFilter onFilter={handleFilter} />
      <PriceSort onSort={handleSort} />
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        {displayed.map((p, i) => <ProductCard key={i} {...p} />)}
      </div>
      <Pagination currentPage={currentPage} totalPages={totalPages}
onPageChange={setCurrentPage} />
    </section>
  );
}

```


Option 2: Infinite Scroll

Update Catalogue.jsx

```

import { useState, useEffect } from "react";
import ProductCard from "../ProductCard";
import ProductFilter from "../ProductFilter";
import PriceSort from "../PriceSort";

const products = [
  // ... full product list
];

export default function Catalogue() {
  const [filtered, setFiltered] = useState(products);
  const [visibleCount, setVisibleCount] = useState(6);

  const handleFilter = ({ search, gauge, finish }) => {
    let result = products;
    if (search) result = result.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
    if (gauge) result = result.filter(p => p.gauge === gauge);
    if (finish) result = result.filter(p => p.finish === finish);
    setFiltered(result);
    setVisibleCount(6);
  };

  const handleSort = (order) => {
    let sorted = [...filtered];
    if (order === "asc") sorted.sort((a, b) => a.price - b.price);
    else if (order === "desc") sorted.sort((a, b) => b.price - a.price);
    setFiltered(sorted);
  };

  useEffect(() => {
    const onScroll = () => {
      if (window.innerHeight + window.scrollY >= document.body.offsetHeight - 200) {
        setVisibleCount(prev => Math.min(prev + 6, filtered.length));
      }
    };
    window.addEventListener("scroll", onScroll);
    return () => window.removeEventListener("scroll", onScroll);
  }, [filtered]);

  return (
    <section className="p-6">
      <ProductFilter onFilter={handleFilter} />
      <PriceSort onSort={handleSort} />
      <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
        {filtered.slice(0, visibleCount).map((p, i) => <ProductCard key={i} {...p} />)}
      </div>
    </section>
  );
}

```

✓ Features Now Covered

- **Search + Filtering** (by name, gauge, finish).
- **Price Sorting** (ascending/descending).
- **Pagination or Infinite Scroll** for scalability.
- **Accessibility**: labels, ARIA attributes, focus styles.
- **WhatsApp-first ordering flow**.

If you want a **cleaner UX for mobile users**, I'd recommend **infinite scroll**. For **desktop contractors comparing products**, **pagination** is clearer.

Do you prefer me to finalize with **infinite scroll** as default, or keep **pagination** for structured browsing?